

PLAN-CHANGES-21082026

Plan — simplify the engine to two doors, one snapshot, homogeneous TDS

Written 2026-08-21. This is a **plan for code & testing changes**, not a description of the current system. The current system still has four dispositions, two run kinds, and mixed-format TDS definitions. *README.md* already describes the target model below; this file is the bridge from the code as it stands to the code *README* now claims.

The repo's thesis is "the engine is small". Three of its current features are not pulling their weight and obscure that thesis. This plan removes them, in a order that keeps `make verify` green at every step.

Context — what is being removed, and why

Three simplifications, each already reflected in *README.md* (Slides 6, 7, 9):

- Two dispositions, not four.** Door today is `WRITTEN / EXCLUDED / REJECTED / DEDUPLICATED`. The balancing equation is `src_read = written + excluded + rejected + deduplicated`. After: `WRITTEN / REJECTED` only, `src_read = written + rejected`. The `filter(exclude)` and `dedup` stages are deleted; the engine becomes `parse → map → schema`. (README Slide 6.)
- One full snapshot per run, no deltas.** Today a run has a `run_kind` (initial/delta) and a `window_from / window_to`; `make run-delta` runs an initial then a delta over a later window. After: every run is one full snapshot of the source, scoped only by `run_id`. No initial/delta distinction, no from/to window. (README Slide 10.)
- Homogeneous TDS definitions.** Today one `Record` definition can mix `DAT` fields (fixed offsets) and `JSON` fields (paths) — `Field.fmt` is a per-field discriminator. After: one definition is one format — either all fixed-offset `.DAT` or all `JSONPath` — never mixed in the same file. (README Slide 8.)

The one design tension this creates (and how it resolves)

Removing dedup removes the old idempotency story ("replays are safe because the dedup key collapses duplicates"). The replacement is already in the code: the **count-then-DELETE per `run_id`** in `pipelines/file_processor/pipeline.py:381-428` (commit `e2ebe53`), which clears a run's rows before re-writing them. With dedup gone, idempotency is "re-run the same `run_id` → it replaces its own rows, never duplicates." No new mechanism is needed; the plan only makes this the *documented* idempotency story and drops the dedup-key one.

Removing dedup also means **duplicate source keys would all be WRITTEN**, which would break the current "every key appears exactly once in TARGET" acceptance criterion. The plan resolves this by a stated

assumption — **the source snapshot contains no duplicate account keys** (duplicates were a harness artifact, `harness/generate.py --duplicates` , not a real mainframe scenario) — and removes the harness's duplicate generation. The reconciliation key, today `dedup_key` , is renamed to the **account key** (`account_key`), the actual source identifier, with no survivor-rank tie-break behind it.

Removing the filter means **CORP/PRIV records (today EXCLUDED by mapping-project1.yaml) now flow through `map` → `schema`** and are either WRITTEN or REJECTED like every other record. That is a deliberate semantic change, not a regression: "excluded by contract" was a policy filter dressed as an engine stage. If a contract genuinely should not migrate CORP/PRIV, that is a source-feed concern (don't extract them), not an engine stage.

Decisions

#	Decision	Rationale
D1	Drop <code>Door.EXCLUDED</code> and <code>Door.DEDUPLICATED</code> , the <code>FILTER</code> and <code>DEDUP</code> stages, and their reasons.	Slide 6. Two doors, two dispositions.
D2	The reconciliation key is <code>account_key</code> (was <code>dedup_key</code>), with no survivor rank.	Dedup is gone; the key is the source identifier, not a tie-break winner.
D3	Idempotency = count-then-DELETE per <code>run_id</code> (already implemented). Dedup-key idempotency is removed from docs and tests.	The replacement already exists and is the only mechanism that survives.
D4	The source snapshot is assumed to contain no duplicate account keys. The harness stops generating duplicates.	Without dedup, duplicates would all write; this is a stated precondition, not an engine guarantee.
D5	Remove <code>run_kind</code> , <code>window_from</code> , <code>window_to</code> from the CLI, orchestrator, DAG, harness, schemas and fixtures.	Slide 9. One full snapshot per run, scoped by <code>run_id</code> only.
D6	A TDS <code>Record</code> definition is homogeneous: all fields <code>DAT</code> or all fields <code>JSON</code> , validated at load. The per-field <code>fmt</code> discriminator is removed.	Slide 8. One format per definition.
D7	Acceptance criteria go from 10 to 8: drop #2 (excluded-exact) and #10 (delta run); rewrite #1, #3, #6, #7.	#2 and #10 assert removed features; the rest reference removed columns.

Component map — every file touched, and how

Engine core

`pipelines/common/doors.py`

- `Door` : keep `WRITTEN` , `REJECTED` ; delete `EXCLUDED` , `DEDUPLICATED` . The `exit` property becomes trivially `MIGRATED` if `WRITTEN` else `NOT_MIGRATED` .
- `Stage` : keep `PARSE` , `MAP` , `SCHEMA` ; delete `FILTER` , `DEDUP` .
- `Reason` : delete `FILTER_EXCLUDED_BY_CONTRACT` , `DEDUP_LOST_SURVIVOR_RANK` . Keep all `PARSE_*` , `MAP_*` , `SCHEMA_INVALID` .

- `Counters`: delete `excluded`, `deduplicated` fields; `count()` handles only `WRITTEN / REJECTED`. `not_migrated` becomes `self.rejected`. `accounted` becomes `written + rejected`. `to_dict()` drops the two keys. `merge()` drops the two adds.
- `BalanceError` message: drop the `excluded / deduplicated` breakdown.
- `Lineage`: rename `dedup_key` → `account_key` (or drop if redundant with `source_key` — see D2; `source_key` already identifies the record, `dedup_key` was the tie-break key). Decision: **drop the `dedup_key` field**; `source_key` is the identity. Update the docstring that references `_dedup_key`.
- Module docstring (lines 1-17): rewrite the four-disposition expansion to the two-door form `src_read = written + rejected`.

`pipelines/common/engine.py`

- `route_intake` (line 46): delete the `if engine.is_excluded(fields): return EXCLUDED` branch (line 88-89). Delete `dedup_key=engine.dedup_key(fields)` from the returned `Outcome` (line 94) — or keep `dedup_key` renamed to `account_key` if `Outcome` still carries it for the writer. Decision: rename to `account_key`, sourced from the mapping's key field (was `dedup_key`).
- `survivor_rank` (line 100): **delete entirely** — no tie-break without dedup.
- `RecordRouter.route` (line 120): delete the `seen` set and the dedup branch (lines 140-172). `route()` becomes `parse` → `map` → `schema` with no `seen` parameter. The `batch_id / seen` signature becomes `route(self, raw, batch_id)`.
- `is_excluded / dedup_key` on `TransformEngine`: `is_excluded` is deleted; `dedup_key` is renamed `account_key` (still derived from the mapping's key fields — the same field, just no longer a *dedup* key).
- Module docstring (lines 1-66): rewrite the stage diagram from `parse` → `filter` → `dedup` → `map` → `schema` to `parse` → `map` → `schema`; remove the "exclusion precedes dedup" paragraph and the Beam-vs-in-memory dedup paragraph.

`pipelines/common/tds.py` (Slide 8 — homogeneous definitions)

- `Field`: remove the `fmt` discriminator (line 35). A field is `DAT` or `JSON` *by the record's layout*, not per-field. Keep `offset / length / col` (`DAT`) and `path` (`JSON`).
- `parse_tds`: add a **homogeneity check** on `flush()` — every field in a record must resolve under the same layout kind (all fixed-offset/col for a `fixed / csv` layout, or all `path`-based for a `json` layout). Mixing raises `ValueError` at definition load.
- `RecordParser.parse`: the `if f.fmt == "JSON"` branch (line 286) is replaced by a layout-kind check — `json` layout resolves all fields by `path`; `fixed / csv` resolve all fields by `offset/col`. No per-field `fmt` switch.
- Module docstring (lines 1-14): rewrite "one definition describes both fixed-offset `.DAT` fields and `JSON` fields" → "one definition is one format — `.DAT` or `JSON`, never mixed in the same file."

File processor (Beam)

`pipelines/file_processor/pipeline.py`

- `LEDGER_SCHEMA` (~line 76): drop `window_from`, `window_to`, `excluded` columns; drop `duplicates` if present. Keep `src_read`, `extraction_written`, `rejected`, `balanced`.
- `account_src` schema (~line 435): drop `_run_kind`, `_window_from`, `_window_to`; rename `_dedup_key` → `_account_key`.
- `record_lineage` schema: drop `dedup_key` (per D2); keep `source_key`.

- `account_target` schema: rename `dedup_key` → `account_key` .
- `ParseAndFilterFn` (line 220) → rename `ParseFn` : delete the `EXCLUDED` tagged output (lines 233, 271-294) and the `c_excluded` counter. Parse failures still go to `rejected` . Emits `(account_key, payload)` on the main output.
- `DedupFn` (line 304): **delete entirely** — the `GroupByKey` dedup stage, the `DUPLICATES` tagged output, the `c_dedup` counter, and the survivor-rank logic. The pipeline no longer groups by key before writing; parsed records go straight to the writer. (The `deduped = PTransform` chain at line 460 is removed; the main output of `ParseFn` feeds the writer directly.)
- `run_pipeline` signature (line 381): drop `run_kind` , `window_from` , `window_to` parameters. The count-then-DELETE block (lines 381-428) **stays** — it is now the idempotency mechanism (D3). Drop the `_run_kind / _window_from / _window_to` fields from the written rows (line 435-437); keep `_account_key` .
- Module docstring (lines 1-30): rewrite "three of the four not-migrated dispositions are settled here — parse, filter and dedup" → "parse is settled here; map and schema happen downstream"; remove the dedup/GroupByKey paragraph.

Harness

`harness/generate.py`

- `Manifest` (lines 72-82): drop `run_kind` , `window_from` , `window_to` , `expected_excluded` , `expected_deduplicated` . Keep `expected_by_reason` . Add nothing — the manifest no longer seeds exclusions or duplicates.
- `generate()` (line 186): drop `run_kind` , `window_from` , `window_to` , `duplicates` , `excluded_ratio` parameters. Drop the `n_excluded / CORP-PRIV` branch (lines 196-210) — all generated accounts are normal records that flow through map/schema. Drop the duplicate-cloning branch (lines 216-225). `base = 0` always (no delta id offset).
- CLI (lines 252-272): drop `--window` , `--window-from` , `--window-to` , `--duplicates` , `--excluded-ratio` . Output path becomes `local/data/mainframe/ACCOUNT.src` (no `initial / delta` subdir) — or keep a single `initial/` dir if other code reads that path; decide during implementation by grepping `local/data/mainframe` .
- The CORP/PRIV comment (line 26) is removed — those client types are no longer special.
- **Concrete fixture numbers** (`harness/README.md`, `tests/README.md`): generated accounts 500 → 400 (the 100 CORP/PRIV exclusions and 20 duplicates are no longer seeded); total records 526 → 406; the manifest count line goes `400 written + 100 excluded + 6 rejected + 20 deduplicated` → `400 written + 6 rejected` ; the balance becomes `406 = 400 + 6` . Both READMEs state these numbers; `generate.py` must produce them and `tests/acceptance.py` must assert them.

Tests

`tests/acceptance.py` — criteria 10 → 8 (D7):

- **#1 balancing**: `not_migrated = row["excluded"] + rejected_total + row["duplicates"]` → `not_migrated = rejected_total` . `accounted = written + not_migrated` . Drop the `excluded/deduplicated` fields from the assertion message.
- **#2 excluded-exact: delete** (D7). Excluded no longer exists.

- **#3 reject_reasons:** unchanged in shape — still compares `reject_log` reason counts to `manifest["expected_by_reason"]`. The manifest still seeds malformed records with the same reason codes; only the set of reasons shrinks (no FILTER/DEDUP reasons).
- **#6 key_level:** rewrite. `dedup_key` → `account_key` in the join. The "exactly once in TARGET" assertion (lines 205-209) holds only because the source has no duplicates (D4); keep it as a source-no-duplicates check, or relax to "orphans = 0" only. Decision: keep orphan check + exactly-once (both hold under D4), rename the column.
- **#7 lineage_named:** `expected` dict drops `excluded` and `deduplicated` doors; only `rejected` remains. The lineage/ledger agreement is now single-door.
- **#10 delta_run: delete** (D7). No deltas. Also delete the `local/state/last_delta_run_id` read (lines 286-290).
- **#4, #5, #8, #9:** unchanged (schema valid, batches, artefacts, checksums).
- Module docstring (lines 1-9): drop "The excluded count must match exactly"; the oracle is still the harness manifest, now for reject reasons only.
- Renumber: criteria become 1-8 contiguous. The mapping (old → new): #3 → #2, #4 → #3, #5 → #4, #6 → #5, #7 → #6, #8 → #7, #9 → #8; #1 stays #1; old #2 (excluded-exact) and #10 (delta run) are deleted. (The current code already numbers 10 before 9 in source order; fix the ordering while here.) Drop `test_dedup_survivor_rule_is_shared_and_replay_stable` and any `test_*excluded*` from `test_engine.py`.

tests/test_engine.py — adjust to the new `route / route_intake` signatures:

- Drop `seen` from `RecordRouter.route` calls; drop tests that assert EXCLUDED or DEDUPLICATED outcomes (e.g. `test_dedup_survivor_rule_is_shared`, any `test_*excluded*`).
- Add a homogeneity test for `parse_tds` (a mixed DAT/JSON record raises `ValueError`).
- The self-comparing oracle fix from the prior session (line 515) stays.

tests/project_smoke.py — drop delta/window references if any (grep confirmed it references the removed concepts).

Orchestrator & DAG

local/scripts/run_pipeline.py

- `run_once` (line 100): drop `run_kind`, `window_from`, `window_to` params; drop the `--window / --window-from / --window-to / --run-kind` args passed to harness, `file_processor`, `enrichment`, `recon` (lines 110-166).
- CLI (line 170): drop `--mode` (initial/delta). `--duplicates` (line 174) is dropped.
- `main` (line 200): drop the delta branch (lines 207-214); one run per invocation. Drop `initial_to / window` plumbing. Drop the `last_delta_run_id` state file logic (lines 220-226).
- `make run-delta` (Makefile line 89-91) is removed; `make run-initial` becomes `make run` (or keep `run-initial` as an alias). `make verify` is unchanged in target.

composer/dags/mig_000001_1.py

- DAG params (lines 124-127): drop `RUN_KIND`, `WINDOW_FROM`, `WINDOW_TO`; keep `RUN_ID`.
- `common_args` (lines 327-332): drop `--run-kind / --window-from / --window-to`.

- `assert_run_balanced` (lines 285-291): query `excluded, rejected, duplicates` → query `rejected` only; `not_migrated = rejected`. Drop the `excluded/duplicates` columns.
- Loader & recon tasks (lines 388-424): drop window args.
- The DAG runs one full-snapshot load per `RUN_ID`; no delta tasks.

Apps (Java)

`apps/common` — `RunContext` (apps/README.md): drop `run_kind`, `window_from`, `window_to` from the run context; stamp `run_id` only. The loader and recon read the run id from it and bind it as a query parameter.

`apps/loader-app` & `apps/recon-service` — CLI: drop the `--run-kind` argument (keep `--run-id`). The DAG task sections above drop the window args they pass to these jars; the jars themselves must stop accepting `--run-kind` / `--window-*`.

The loader's `dedupKey` / `X-Idempotency-Key` is **NOT renamed**. It is a separate, retained Target-System idempotency concept (the loader de-duplicates its own POSTs to Target System against the stand-in). D2 renames the *engine* reconciliation key `dedup_key` → `account_key`; it must not touch the loader's `dedupKey` field or header. (apps/README.md calls this out explicitly.)

Contracts

`contracts/mappings/mapping-project1.yaml`

- Delete the `filters: exclude: block ( field: CLIENT_TYPE in: [CORP, PRIV] )`.
- Rename `dedup: key: [ACCT_ID]` → `key: [ACCT_ID]` (it is now the account key, not a dedup key) — or remove the `dedup: wrapper` and surface `key` at top level. The mapping engine reads this as the account key for `account_key` derivation and reconciliation.
- TDS definitions referenced here: ensure each is homogeneous (all `.DAT` or all JSON) per D6. If `mapping-project1.yaml` points at a mixed definition, split it.

Dataform

`dataform/` — grep for `dedup_key`, `excluded`, `duplicates`, `window_from`, `window_to`, `run_kind` in the SQLX. Concretely (dataform/README.md):

- `run_ledger` and `account_target` views: drop `excluded`, `duplicates`, `window_from`, `window_to`, `run_kind`; rename `dedup_key` → `account_key`. Reconciliation SQL that joins on `dedup_key` joins on `account_key` instead.
 - `account_age_years`: today derived as a date difference against the **run window**; after, derived against the **run's snapshot date** (one snapshot per run, no window).
 - Table partitioning: `partitionBy` on `run_id` instead of run window; cluster by `account_key` (was `dedup_key`). production-readiness.md §2.2 states this for `account_src` / `account_curated`.
 - Any view keyed on the run window's from/to bounds is re-scoped to `run_id` only.
-

Build sequence (each step leaves `make verify` green)

1. **Renamed key, no behavior change.** Rename `dedup_key` → `account_key` across `doors.py`, `engine.py`, `file_processor/pipeline.py` schemas, `dataform/SQLX`, `tests/acceptance.py` #6, `mapping-project1.yaml`. Re-run tests. (Pure rename; green.)
2. **Drop the filter stage.** Delete `is_excluded`, the `EXCLUDED` branch in `route_intake`, the `EXCLUDED` tagged output in `ParseAndFilterFn`, `Door.EXCLUDED`, `Stage.FILTER`, `Reason.FILTER_EXCLUDED_BY_CONTRACT`, the `excluded` counter/ledger column, the mapping's `filters.exclude`, harness `excluded_ratio` /CORP-PRIV generation, manifest `expected_excluded`, acceptance #2. CORP/PRIV records now flow through. Re-run tests — acceptance #1, #7 adjust to drop `excluded`. (Green with 9 criteria.)
3. **Drop the dedup stage.** Delete `DedupFn`, `survivor_rank`, the `seen` set in `RecordRouter`, `Door.DEDUPLICATED`, `Stage.DEDUP`, `Reason.DEDUP_LOST_SURVIVOR_RANK`, the `duplicates` counter/ledger column, harness `--duplicates` generation, manifest `expected_deduplicated`, the dedup tests. The count-then-DELETE stays as idempotency. Re-run tests — acceptance #1, #7 adjust to drop `duplicates`. (Green with 8 criteria.)
4. **Drop deltas & windows.** Remove `run_kind / window_from / window_to` from `run_pipeline.py`, the file processor, the DAG, harness, schemas, fixtures. Remove `make run-delta`, `--mode`, the delta branch, `last_delta_run_id`, acceptance #10. (Green with 8 criteria, one run kind.)
5. **Homogeneous TDS.** Remove `Field.fmt`, add the homogeneity check in `parse_tds`, simplify `RecordParser.parse` to branch on layout kind. Split any mixed definition in the contracts. Add the homogeneity test. (Green.) This step also lands the **pipe-delimited-with-header source .DAT** (the target source feed) and the two explicit strip transforms (`strip_trailing_spaces`, `strip_leading_zeros`) specified in `contracts/README.md` §"Source .DAT physical format (target)".
6. **Doc sweep.** The markdown is already swept (uncommitted) to describe the target model and point here for the code bridge — `README.md`, `ARCHITECTURE.md`, `contracts/README.md`, the six component READMEs, and the docs/ files (production-readiness, runbook-gcp, alternative-implementations, PLAN, evidence-map). The sweep also documents the reconciliation-with-Target-System return edge (README Slide 7, `ARCHITECTURE.md` C3) as a future production edge. Historical evidence under `docs/evidence/` is left as-is. This step lands those docs with the code.

Steps 2 and 3 each remove one disposition and its acceptance criterion; doing them separately keeps each diff reviewable and each `make verify` green. If preferred, they can be collapsed into one "four doors → two doors" step.

Verification

- `make verify` after each step: 10 → 9 → 8 → 8 → 8 → 8 criteria, all green.
- `make run` (was `run-initial`) then `make verify` against the live local stack.
- Replay idempotency: re-run the same `run_id`, assert `account_target` row count is unchanged (the count-then-DELETE replaces, never duplicates) — this is the new acceptance-level proof of D3, optionally added as an explicit criterion.
- `make smoke-gcp` on GCP: one full-snapshot DAG run, `run_ledger` balanced as `src_read = written + rejected`, no `excluded / duplicates / window_*` columns.
- Homogeneity: a TDS definition mixing DAT and JSON fields in one record fails to load.

Out of scope

- The Beam pipeline structure outside the removed stages (enrichment, json_producer, loader, recon) is unchanged.
- The dual-writer seam (`FileLoadsBigQueryWriter` / `InsertAllBigQueryWriter`) is untouched. The seam is real and in the right place: at the 20B-row target `insertAll` is the wrong sink (quota-limited, per-row priced, streaming), so `FILE_LOADS` via load jobs is the right destination; the missing piece is the `FILE_LOADS` *implementation* behind the seam, scheduled against the first real-volume test, not this plan. (Was `docs/SIMPLIFICATIONS.md` proposal #3, inlined here before that file was deleted.)
- The shared-task-list proposal is orthogonal: `local/scripts/run_pipeline.py` and `composer/dags/mig_000001_1.py` encode the same seven-stage graph twice, in two idioms; extracting it to one declarative list both consume is the highest-value cleanup left, but this plan does not collapse the two orchestrators — it only removes delta/window from both. (Was `docs/SIMPLIFICATIONS.md` proposal #1, inlined here before that file was deleted.)
- The reconciliation-with-Target-System return edge (README Slide 7, ARCHITECTURE.md C3 — Target System publishes confirmation/audit streams the recon service would consume) is a documented future production edge, not a simplification step; it is not implemented by this plan.
- Historical evidence bundles under `docs/evidence/` record past runs with the old four-disposition, delta model; they are left as historical records, not rewritten.